

AMRITA VISHWA VIDYAPEETHAM CHENNAI
CAMPUS

Amrita School of Engineering

Department of ECE

Coding Practice

Activity Sheets

(For the Batch 2023-2027)

Name : N.Lokeshwar Varma
Roll No. : CH.EN.U4CCE24020
Department: CCE

Instructions

The **Mock Test syllabus** for CSE CT | CYS | AIE | RAI program is enclosed in the attached document for your reference.

Materials for the Mock Test [Must-DO Coding Questions] is provided in a link.

It is mandatory to upload the **completed Activity sheets** before the deadline in the MS-Form link shared by your class Advisor.

Activity Sheet: 01 - Coding Questions from Arrays – Part 2 [Medium + Hard Questions]

Each Activity Sheets contains total of **20 Questions**.

Platform: Any Online Compiler of your choice – **GDB Compiler**

Language: C | C++ | JAVA | PYTHON

The Activity Sheet will have the coding Questions and the Test Case. Once executed in Online Compiler, take **screen shot of the code and the output** and paste it in the space provided in the activity sheet.

The Final Activity Sheet should be in **PDF Format** and the file name should be your **roll number**.

Course Instructors: CARE TEAM

Dr.R.Annamalai Assistant Professor (Sl.Grade) | CSE-AIE

Dr.J.Umameswaran Assistant Professor (Sr.Grade) | CSE-CT

Ms.D. Sasikala Assistant Professor | CSE-AIE

Mr.NirmalRaj Assistant Professor | CSE-CYS

1. You are given a function,

```
int findCount(int arr[], int length, int num, int diff);
```

The function accepts an integer array 'arr', its length and two integer variables 'num' and 'diff'.

Implement this function to find and return the number of elements of 'arr' having an absolute difference of less than or equal to 'diff' with 'num'.

Note: In case there is no element in 'arr' whose absolute difference with 'num' is less than or equal to 'diff', return -1.

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int findCount(int arr[], int length, int num, int diff)
5  {
6      int count = 0;
7
8      for(int i = 0; i < length; i++)
9      {
10         if(abs(arr[i] - num) <= diff)
11         {
12             count++;
13         }
14     }
15
16     if(count == 0)
17     {
18         return -1;
19     }
20
21     return count;
22 }
23
24 int main()
25 {
26     int length, num, diff;
27
28     printf("Roll Number : CH.EN.U4CCE24020\n\n");
29
30     printf("Enter the size of array: ");
31     scanf("%d", &length);
32
33     int arr[length];
34
35     printf("Enter the array elements:\n");
36
37     for(int i = 0; i < length; i++)
38     {
39         scanf("%d", &arr[i]);
40     }
41
42     printf("Enter the number: ");
43     scanf("%d", &num);
44
45     printf("Enter the difference value: ");
46     scanf("%d", &diff);
47
48     int result = findCount(arr, length, num, diff);
49
50     printf("\nOutput: %d\n", result);
51
52     return 0;
53 }
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter the size of array: 5
Enter the array elements:
2
5
8
3
1
Enter the number: 2
Enter the difference value: 1

Output: 3

-----
Process exited after 24.4 seconds with return value 0
Press any key to continue . . . |
```

2. You are required to implement the following Function `def LargeSmallSum(arr)`

The function accepts an integers `arr` of size '`length`' as its arguments you are required to return the sum of second largest element from the even positions and second smallest from the odd position of given '`arr`'

Assumption:

- All array elements are unique
- Treat the 0th position as even

NOTE

- Return 0 if array is empty
- Return 0, if array length is 3 or less than 3

Code:

```

1 //Function to return the sum of second largest element from the even positions and second smallest from the odd position of given 'arr'
2
3 int LargeSmallSum(int arr[], int length)
4 {
5     if(length <= 3)
6         return 0;
7
8     int even[length], odd[length];
9     int n = 0, m = 0;
10
11     for(int i = 0; i < length; i++)
12     {
13         if(i % 2 == 0)
14             even[n++] = arr[i];
15         else
16             odd[m++] = arr[i];
17     }
18
19     for(int i = 0; i < n - 1; i++)
20     {
21         for(int j = i + 1; j < n; j++)
22         {
23             if(even[i] > even[j])
24             {
25                 int temp = even[i];
26                 even[i] = even[j];
27                 even[j] = temp;
28             }
29         }
30     }
31
32     for(int i = 0; i < m - 1; i++)
33     {
34         for(int j = i + 1; j < m; j++)
35         {
36             if(odd[i] < odd[j])
37             {
38                 int temp = odd[i];
39                 odd[i] = odd[j];
40                 odd[j] = temp;
41             }
42         }
43     }
44
45     return even[n-1] + odd[1];
46 }
47
48 //Driver Code
49 int main()
50 {
51     int length;
52     printf("Enter the size of array: ");
53     scanf("%d", &length);
54
55     int arr[length];
56     printf("Enter the array elements:\n");
57
58     for(int i = 0; i < length; i++)
59     {
60         scanf("%d", &arr[i]);
61     }
62
63     int result = LargeSmallSum(arr, length);
64     printf("\nOutput: %d\n", result);
65     return 0;
66 }

```

Output

```
Roll Number : CH.EN.U4CCE24020
```

```
Enter the size of array: 6
```

```
Enter the array elements:
```

```
2
```

```
8
```

```
4
```

```
6
```

```
3
```

```
1
```

```
Output: 9
```

```
-----
```

```
Process exited after 15.83 seconds with return value 0
```

```
Press any key to continue . . . |
```

3. You are required to input the size of the matrix then the elements of matrix, then you have to divide the main matrix in two sub matrices (even and odd) in such a way that element at 0 index will be considered as even and element at 1st index will be considered as odd and so on. then you have sort the even and odd matrices in ascending order then print the sum of second largest number from both the matrices

Code:

```

1 //Solution: solution-3.js
2
3 // Main function
4 function main() {
5     // Get input
6     let rows, cols;
7     print("Enter Number of Rows: ");
8     rows = parseInt(readLine());
9     print("Enter Number of Columns: ");
10    cols = parseInt(readLine());
11
12    // Create matrix
13    let matrix = new Array(rows).fill(0).map(() => new Array(cols).fill(0));
14
15    // Get matrix elements
16    print("Enter matrix elements:");
17    for (let i = 0; i < rows; i++) {
18        for (let j = 0; j < cols; j++) {
19            matrix[i][j] = parseInt(readLine());
20        }
21    }
22
23    // Split matrix into even and odd matrices
24    let even = new Array(rows).fill(0).map(() => new Array(cols).fill(0));
25    let odd = new Array(rows).fill(0).map(() => new Array(cols).fill(0));
26
27    // Split even matrix
28    for (let i = 0; i < rows; i++) {
29        for (let j = 0; j < cols; j++) {
30            if (i % 2 === 0) {
31                even[i][j] = matrix[i][j];
32            } else {
33                odd[i][j] = matrix[i][j];
34            }
35        }
36    }
37
38    // Sort even matrix
39    for (let i = 0; i < rows; i++) {
40        for (let j = 0; j < cols; j++) {
41            let temp = even[i][j];
42            let k = j + 1;
43            while (k < cols) {
44                if (even[i][k] < temp) {
45                    [temp, even[i][k]] = [even[i][k], temp];
46                }
47                k++;
48            }
49        }
50    }
51
52    // Sort odd matrix
53    for (let i = 0; i < rows; i++) {
54        for (let j = 0; j < cols; j++) {
55            let temp = odd[i][j];
56            let k = j + 1;
57            while (k < cols) {
58                if (odd[i][k] < temp) {
59                    [temp, odd[i][k]] = [odd[i][k], temp];
60                }
61                k++;
62            }
63        }
64    }
65
66    // Get second largest
67    let secondLargestEven = even[0][rows - 2];
68    let secondLargestOdd = odd[0][rows - 2];
69
70    // Sum
71    let sum = secondLargestEven + secondLargestOdd;
72
73    print("Second Largest in Even Matrix: " + secondLargestEven);
74    print("Second Largest in Odd Matrix: " + secondLargestOdd);
75    print("Sum: " + sum);
76
77    return 0;
78 }

```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter number of rows: 4
Enter number of columns: 2
Enter matrix elements:
1
5
6
4
8
9
2
3

Second Largest in Even Matrix: 6
Second Largest in Odd Matrix: 5
Sum: 11

-----
Process exited after 13.03 seconds with return value 0
Press any key to continue . . .
```


4. Given an integer array Arr of size N the task is to find the count of elements whose value is greater than all of its prior elements.

Note : 1st element of the array should be considered in the count of the result.

For example,

Arr[]={7,4,8,2,9}

As 7 is the first element, it will consider in the result.

8 and 9 are also the elements that are greater than all of its previous elements.

Since total of 3 elements is present in the array that meets the condition.

Hence the output = 3.

Code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int N;
6
7      printf("Roll Number : CH.EN.U4CCE24020\n\n");
8
9      printf("Enter the size of array: ");
10     scanf("%d", &N);
11
12     int Arr[N];
13
14     printf("Enter the array elements:\n");
15
16     for(int i = 0; i < N; i++)
17     {
18         scanf("%d", &Arr[i]);
19     }
20
21     int count = 1;
22     int max = Arr[0];
23
24     for(int i = 1; i < N; i++)
25     {
26         if(Arr[i] > max)
27         {
28             count++;
29             max = Arr[i];
30         }
31     }
32
33     printf("\nCount of elements greater than all previous elements: %d\n", count);
34
35     return 0;
36 }
```

Output

```
Roll Number : CH.EN.U4CCE24020  
  
Enter the size of array: 4  
Enter the array elements:  
3  
9  
12  
1  
  
Count of elements greater than all previous elements: 3  
  
-----  
Process exited after 7.211 seconds with return value 0  
Press any key to continue . . .
```

5. At a fun fair, a street vendor is selling different colours of balloons. He sells N number of different colours of balloons (B[]). The task is to find the colour (odd) of the balloon which is present odd number of times in the bunch of balloons.

Note: If there is more than one colour which is odd in number, then the first colour in the array which is present odd number of times is displayed. The colours of the balloons can all be either upper case or lower case in the array. If all the inputs are even in number, display the message “All are even”.

Code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int N;
6
7      printf("Roll Number : CH.EN.U4CCE24020\n\n");
8
9      printf("Enter the number of balloons: ");
10     scanf("%d", &N);
11
12     char B[N];
13
14     printf("Enter the balloon colours:\n");
15
16     for(int i = 0; i < N; i++)
17     {
18         scanf("%c", &B[i]);
19     }
20
21     int found = 0;
22
23     for(int i = 0; i < N; i++)
24     {
25         int count = 0;
26
27         for(int j = 0; j < N; j++)
28         {
29             if(B[i] == B[j])
30             {
31                 count++;
32             }
33         }
34
35         if(count % 2 != 0)
36         {
37             printf("\nOdd colour balloon: %c\n", B[i]);
38             found = 1;
39             break;
40         }
41     }
42
43     if(found == 0)
44     {
45         printf("\nAll are even\n");
46     }
47
48     return 0;
49 }
```

Output

```
Roll Number : CH.EN.U4CCE24020
```

```
Enter the number of balloons: 5
```

```
Enter the balloon colours:
```

```
red
```

```
blue
```

```
Odd colour balloon: r
```

```
-----
```

```
Process exited after 13.06 seconds with return value 0
```

```
Press any key to continue . . .
```

6. There are total n number of Monkeys sitting on the branches of a huge Tree. As travelers offer Bananas and Peanuts, the Monkeys jump down the Tree. If every Monkey can eat k Bananas and j Peanuts. If total m number of Bananas and p number of Peanuts are offered by travelers, calculate how many Monkeys remain on the Tree after some of them jumped down to eat.

At a time one Monkey gets down and finishes eating and go to the other side of the road. The Monkey who climbed down does not climb up again after eating until the other Monkeys finish eating.

Monkey can either eat k Bananas or j Peanuts. If for last Monkey there are less than k Bananas left on the ground or less than j Peanuts left on the ground, only that Monkey can eat Bananas($<k$) along with the Peanuts($\leq j$).

Write code to take inputs as n , m , p , k , j and return the number of Monkeys left on the Tree.

Where, n = Total no of Monkeys

k = Number of eatable Bananas by Single Monkey (Monkey that jumped down last may get less than k Bananas)

j = Number of eatable Peanuts by single Monkey (Monkey that jumped down last may get less than j Peanuts)

m = Total number of Bananas

p = Total number of Peanuts

Remember that the Monkeys always eat Bananas and Peanuts, so there is no possibility of k and j having a value zero

Code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int n, m, p, k, j;
6
7      printf("Roll Number : CH.EN.U4CCE24020\n\n");
8
9      printf("Enter total number of monkeys: ");
10     scanf("%d", &n);
11
12     printf("Enter total number of bananas: ");
13     scanf("%d", &m);
14
15     printf("Enter total number of peanuts: ");
16     scanf("%d", &p);
17
18     printf("Enter number of bananas eaten by one monkey: ");
19     scanf("%d", &k);
20
21     printf("Enter number of peanuts eaten by one monkey: ");
22     scanf("%d", &j);
23
24     int monkeysBanana = m / k;
25
26     if(m % k != 0)
27     {
28         monkeysBanana++;
29     }
30
31     int monkeysPeanut = p / j;
32
33     if(p % j != 0)
34     {
35         monkeysPeanut++;
36     }
37
38     int totalMonkeysDown = monkeysBanana + monkeysPeanut;
39
40     int monkeysLeft = n - totalMonkeysDown;
41
42     if(monkeysLeft < 0)
43     {
44         monkeysLeft = 0;
45     }
46
47     printf("\nMonkeys left on the tree: %d\n", monkeysLeft);
48
49     return 0;
50 }
```

Output

```
Roll Number : CH.EN.U4CCE24020

Enter total number of monkeys: 5
Enter total number of bananas: 3
Enter total number of peanuts: 2
Enter number of bananas eaten by one monkey: 2
Enter number of peanuts eaten by one monkey: 3

Monkeys left on the tree: 2

-----
Process exited after 12 seconds with return value 0
Press any key to continue . . .
```

7. Mahesh and Suresh are playing a new game “Checkers“. This is a very simple game but becomes challenging when more expert players are playing. Below is the description of the game and rules: The game is played by 2 players. This game consists of an $N \times M$ matrix. Each of the cells is background lit by lights. And these cells are either Green or Black. The green and black cells are randomly lit and will be represented with 1's and 0's respectively. Green cells are the cells that need to be captured. Black cells cannot be captured. Everyone is in the race to capture the maximum number of cells possible. In a single chance, a player can capture all those adjacent cells which share an edge. Once there is no adjacent edge the chance breaks and the next player will play. Mahesh always starts the game and Suresh is second. Both players are playing optimally, find out how many cells Suresh captures.

Code:

```

#include <iostream>
using namespace std;

void sortDescending(int arr[], int n)
{
    for(int i = 0; i < n - 1; i++)
    {
        for(int j = i + 1; j < n; j++)
        {
            if(arr[i] < arr[j])
            {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}

void dfs(int mat[100][100], int visited[100][100], int n, int m, int i, int j, int *count)
{
    if(i < 0 || j < 0 || i >= n || j >= m)
        return;

    if(mat[i][j] == 0 || visited[i][j] == 1)
        return;

    visited[i][j] = 1;
    (*count)++;

    dfs(mat, visited, n, m, i + 1, j, count);
    dfs(mat, visited, n, m, i - 1, j, count);
    dfs(mat, visited, n, m, i, j + 1, count);
    dfs(mat, visited, n, m, i, j - 1, count);
}

int main()
{
    int n, m;

    cout << "Roll Number : CH.EN.UAC2202W@viva";

    cout << "Enter number of Rows: ";
    cin >> n;

    cout << "Enter number of columns: ";
    cin >> m;

    int mat[100][100];
    int visited[100][100] = {0};

    cout << "Enter matrix elements (0 and 1)";

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            cout << "Enter: ";
            cin >> mat[i][j];
        }
    }

    int groups[100];
    int n = 0;

    // Find connected groups of 1's
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            if(mat[i][j] == 1 && !visited[i][j])
            {
                int count = 0;
                cout << "Group: ";
                dfs(mat, visited, n, m, i, j, &count);
                groups[n++] = count;
            }
        }
    }

    // Sort groups in descending order
    sortDescending(groups, n);

    int result = 0;

    // Remove white (0's) from given matrix
    for(int i = 0; i < n; i++)
    {
        result += groups[i];
    }

    cout << "Cells captured by Suresh: " << result << endl;
    return 0;
}

```


Output:

```
Roll Number : CH.EN.U4CCE24020
```

```
Enter number of rows: 6
```

```
Enter number of columns: 2
```

```
Enter matrix elements (0 and 1):
```

```
0
```

```
0
```

```
0
```

```
1
```

```
1
```

```
1
```

```
1
```

```
0
```

```
0
```

```
1
```

```
0
```

```
1
```

```
Cells captured by Suresh: 2
```

```
-----
```

```
Process exited after 31.99 seconds with return value 0
```

```
Press any key to continue . . .
```

8. A chocolate factory is packing chocolates into packets. The chocolate packets here represent an array `arrt` of `N` number of integer values. The task is to find the empty packets(0) of chocolate and push it to the end of the conveyor belt(array).

For Example:

- `N=7` and `arr = [4,5,0,1,9,0,5,0]`.
- There are 3 empty packets in the given set. These 3 empty packets represented as 0 should be pushed towards the end of the array

Code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int N;
6
7      printf("Roll Number : CH.EN.U4CCE24020\n\n");
8
9      printf("Enter the number of packets: ");
10     scanf("%d", &N);
11
12     int arr[N];
13
14     printf("Enter the packet values:\n");
15
16     for(int i = 0; i < N; i++)
17     {
18         scanf("%d", &arr[i]);
19     }
20
21     int result[N];
22     int k = 0;
23
24     // Store non-zero elements
25     for(int i = 0; i < N; i++)
26     {
27         if(arr[i] != 0)
28         {
29             result[k++] = arr[i];
30         }
31     }
32
33     // Add zeros at the end
34     while(k < N)
35     {
36         result[k++] = 0;
37     }
38
39     printf("\nArray after pushing empty packets to the end:\n");
40
41     for(int i = 0; i < N; i++)
42     {
43         printf("%d ", result[i]);
44     }
45
46     printf("\n");
47
48     return 0;
49 }
```

Output:

```
Roll Number : CH.EN.U4CCE24020
```

```
Enter the number of packets: 4
```

```
Enter the packet values:
```

```
20
```

```
35
```

```
6
```

```
14
```

```
Array after pushing empty packets to the end:
```

```
20 35 6 14
```

```
-----
```

```
Process exited after 12.23 seconds with return value 0
```

```
Press any key to continue . . .
```

9. Given N gold wires, each wire has a length associated with it. At a time, only two adjacent small wires are assembled at the end of a large wire and the cost of forming is the sum of their length. Find the minimum cost when all wires are assembled to form a single wire.

Code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int N;
6
7      printf("Roll Number : CH.EN.U4CCE24020\n\n");
8
9      printf("Enter the number of gold wires: ");
10     scanf("%d", &N);
11
12     int arr[N];
13
14     printf("Enter the lengths of wires:\n");
15
16     for(int i = 0; i < N; i++)
17     {
18         scanf("%d", &arr[i]);
19     }
20
21     int cost = 0;
22
23     while(N > 1)
24     {
25         int minIndex = 0;
26
27         // Find adjacent pair with minimum sum
28         for(int i = 0; i < N - 1; i++)
29         {
30             if(arr[i] + arr[i + 1] < arr[minIndex] + arr[minIndex + 1])
31             {
32                 minIndex = i;
33             }
34         }
35
36         int sum = arr[minIndex] + arr[minIndex + 1];
37
38         cost += sum;
39
40         arr[minIndex] = sum;
41
42         // Shift array left
43         for(int i = minIndex + 1; i < N - 1; i++)
44         {
45             arr[i] = arr[i + 1];
46         }
47
48         N--;
49     }
50
51     printf("\nMinimum cost to assemble all wires: %d\n", cost);
52
53     return 0;
54 }
```

Output:

```
Roll Number : CH.EN.U4CCE24020

Enter the number of gold wires: 5
Enter the lengths of wires:
23
52
10
6
34

Minimum cost to assemble all wires: 266

-----
Process exited after 28.35 seconds with return value 0
Press any key to continue . . . |
```

10. Given an array `Arr[]` of size `T`, contains binary digits, where

- 0 represents a biker running to the north.
- 1 represents a biker running to the south.

The task is to count crossing biker in such a way that each pair of crossing biker(`N`, `S`), where $0 \leq N < S < T$, is passing when `N` is running to the north and `S` is running to the south.

Constraints:

$0 \leq N < S < T$

Code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int T;
6
7      printf("Roll Number : CH.EN.U4CCE24020\n\n");
8
9      printf("Enter the size of array: ");
10     scanf("%d", &T);
11
12     int Arr[T];
13
14     printf("Enter binary digits (0 or 1):\n");
15
16     for(int i = 0; i < T; i++)
17     {
18         scanf("%d", &Arr[i]);
19     }
20
21     int northCount = 0;
22     int crossingBikers = 0;
23
24     for(int i = 0; i < T; i++)
25     {
26         if(Arr[i] == 0)
27         {
28             northCount++;
29         }
30         else if(Arr[i] == 1)
31         {
32             crossingBikers += northCount;
33         }
34     }
35
36     printf("\nNumber of crossing bikers: %d\n", crossingBikers);
37
38     return 0;
39 }
```

Output:

```
Roll Number : CH.EN.U4CCE24020
```

```
Enter the size of array: 6
```

```
Enter binary digits (0 or 1):
```

```
0
```

```
0
```

```
1
```

```
1
```

```
1
```

```
0
```

```
Number of crossing bikers: 6
```

```
-----
```

```
Process exited after 9.441 seconds with return value 0
```

```
Press any key to continue . . .
```

Topic: Arrays – II [Medium + Hard Coding Questions]

11. The program must accept an integer matrix of size $R \times C$ containing unique integers as the input. The program must remove the minimum integer in each row of the matrix. Then the program must remove the minimum integer in each column of the matrix. Finally, the program must print the modified matrix as the output.

Boundary Condition(s):

$3 \leq R, C \leq 50$

$1 \leq \text{Matrix element value} \leq 10^4$

Input Format:

The first line contains R and C separated by a space.

The next R lines, each contains C integers separated by a space.

Output Format:

The first $R-1$ lines, each contains $C-1$ integers separated by a space.

Code:

```

1 // Solution 11
2
3 int main()
4 {
5     int R, C;
6     printf("Enter Number : R, C (R, C <= 50)\n");
7     printf("Enter number of rows and columns: ");
8     scanf("%d %d", &R, &C);
9
10    int matrix[R][C];
11    printf("Enter matrix elements:\n");
12
13    for(int i = 0; i < R; i++)
14    {
15        for(int j = 0; j < C; j++)
16        {
17            scanf("%d", &matrix[i][j]);
18        }
19    }
20
21    int rowToRemove[R];
22
23    // Find minimum element index for each row
24    for(int i = 0; i < R; i++)
25    {
26        int min = matrix[i][0];
27        int index = 0;
28
29        for(int j = 1; j < C; j++)
30        {
31            if(matrix[i][j] < min)
32            {
33                min = matrix[i][j];
34                index = j;
35            }
36        }
37        rowToRemove[i] = index;
38    }
39
40    int temp[C-1];
41
42    // Remove minimum element from each row
43    for(int i = 0; i < R; i++)
44    {
45        int k = 0;
46
47        for(int j = 0; j < C; j++)
48        {
49            if(j != rowToRemove[i])
50            {
51                temp[k++] = matrix[i][j];
52            }
53        }
54    }
55
56    int colToRemove[C-1];
57
58    // Find minimum element index for each column
59    for(int j = 0; j < C-1; j++)
60    {
61        int min = temp[j][0];
62        int index = 0;
63
64        for(int i = 1; i < R; i++)
65        {
66            if(temp[i][j] < min)
67            {
68                min = temp[i][j];
69                index = i;
70            }
71        }
72        colToRemove[j] = index;
73    }
74
75    printf("Modified Matrix:\n");
76
77    // Remove minimum element from each column
78    for(int i = 0; i < R; i++)
79    {
80        int printCol = 0;
81
82        for(int j = 0; j < C-1; j++)
83        {
84            if(j != colToRemove[j])
85            {
86                printf("%d ", temp[i][j]);
87                printCol++;
88            }
89        }
90        printf("\n");
91    }
92
93    return 0;
94 }

```


Output:

```
Roll Number : CH.EN.U4CCE24020

Enter number of rows and columns: 4
3
Enter matrix elements:
2
8
9
4
3
1
4
2
5
0
5

3

Modified Matrix:
8 9
4 5
5 3

-----
Process exited after 27.65 seconds with return value 0
Press any key to continue . . .
```

12. The program must accept two arrays of size N and then for each element position, the program must print find

the sum of odd elements till that position for both the arrays and then print the sum which is higher.

Boundary Condition(s):

1 $\leq$ N $\leq$ 100

1 $\leq$ Each integer value $\leq$ 10⁵

Input Format:

The first line contains N.

The second line contains N integers representing the first array separated by a space.

The third line contains N integers representing the second array separated by a space.

Output Format:

The first line contains N integer values based on the given conditions.

Code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int N;
6
7      printf("Roll Number : CH.EN.U4CCE24020\n\n");
8
9      printf("Enter the size of arrays: ");
10     scanf("%d", &N);
11
12     int arr1[N], arr2[N];
13
14     printf("Enter elements of first array:\n");
15
16     for(int i = 0; i < N; i++)
17     {
18         scanf("%d", &arr1[i]);
19     }
20
21     printf("Enter elements of second array:\n");
22
23     for(int i = 0; i < N; i++)
24     {
25         scanf("%d", &arr2[i]);
26     }
27
28     int sum1 = 0, sum2 = 0;
29
30     printf("\nOutput:\n");
31
32     for(int i = 0; i < N; i++)
33     {
34         if(arr1[i] % 2 != 0)
35         {
36             sum1 += arr1[i];
37         }
38
39         if(arr2[i] % 2 != 0)
40         {
41             sum2 += arr2[i];
42         }
43
44         if(sum1 > sum2)
45         {
46             printf("%d ", sum1);
47         }
48         else
49         {
50             printf("%d ", sum2);
51         }
52     }
53
54     printf("\n");
55
56     return 0;
57 }
```

Output:

```
Roll Number : CH.EN.U4CCE24020
Enter the size of arrays: 3
Enter elements of first array:
0
8
5
Enter elements of second array:
8

5
9

Output:
0 5 14

-----
Process exited after 19 seconds with return value 0
Press any key to continue . . .
```

13. Given an array `nums`, you can perform the following operation any number of times:

Select the adjacent pair with the minimum sum in `nums`. If multiple such pairs exist, choose the leftmost one.

Replace the pair with their sum.

Return the minimum number of operations needed to make the array non-decreasing.

An array is said to be non-decreasing if each element is greater than or equal to its previous element (if it exists). Example 1:

Input: `nums = [5,2,3,1]`

Output: 2

Explanation:

The pair (3,1) has the minimum sum of 4. After replacement, `nums = [5,2,4]`.

The pair (2,4) has the minimum sum of 6. After replacement, `nums = [5,6]`.

The array `nums` became non-decreasing in two operations.

Code:

```

1 //include <stdio.h>
2
3 int isNonDecreasing(int arr[], int n)
4 {
5     for(int i = 1; i < n; i++)
6     {
7         if(arr[i] < arr[i - 1])
8             return 0;
9     }
10    return 1;
11}
12
13 int main()
14 {
15     int n;
16
17     printf("Roll Number : OJEN.DACC240204\n");
18     printf("Enter the size of array: ");
19     scanf("%d", &n);
20
21     int nums[100];
22
23     printf("Enter the array elements:\n");
24     for(int i = 0; i < n; i++)
25     {
26         scanf("%d", &nums[i]);
27     }
28
29     int operations = 0;
30
31     while(!isNonDecreasing(nums, n))
32     {
33         int minsum = nums[0] + nums[1];
34         int index = 0;
35
36         // Find leftmost adjacent pair with minimum sum
37         for(int i = 1; i < n - 1; i++)
38         {
39             int currentsum = nums[i] + nums[i + 1];
40             if(currentsum < minsum)
41             {
42                 minsum = currentsum;
43                 index = i;
44             }
45         }
46
47         // Replace pair with sum
48         nums[index] = nums[index] + nums[index + 1];
49
50         // Shift array left
51         for(int i = index + 1; i < n - 1; i++)
52         {
53             nums[i] = nums[i + 1];
54         }
55
56         n--;
57         operations++;
58     }
59
60     printf("\nMinimum number of operations: %d\n", operations);
61
62     return 0;
63 }

```

Output:

```
Roll Number : CH.EN.U4CCE24020

Enter the size of array: 5
Enter the array elements:
2
3
8
1
0

Minimum number of operations: 3

-----
Process exited after 9.44 seconds with return value 0
Press any key to continue . . . |
```

14. Given a number x and an array of integers arr , find the smallest subarray with sum greater than the given value. If such a subarray do not exist return 0 in that case.

Examples:

Input: $x = 51$, $arr[] = [1, 4, 45, 6, 0, 19]$

Output: 3

Explanation: Minimum length subarray is $[4, 45, 6]$

Input: $x = 100$, $arr[] = [1, 10, 5, 2, 7]$

Output: 0

Explanation: No subarray exist

Constraints:

$1 \leq arr.size, x \leq 10^5$

$0 \leq arr[] \leq 10^4$

Code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int n, x;
6
7      printf("Roll Number : CH.EN.U4CCE24020\n\n");
8
9      printf("Enter the size of array: ");
10     scanf("%d", &n);
11
12     int arr[n];
13
14     printf("Enter the array elements:\n");
15
16     for(int i = 0; i < n; i++)
17     {
18         scanf("%d", &arr[i]);
19     }
20
21     printf("Enter the value of x: ");
22     scanf("%d", &x);
23
24     int minLength = n + 1;
25
26     for(int i = 0; i < n; i++)
27     {
28         int sum = 0;
29
30         for(int j = i; j < n; j++)
31         {
32             sum += arr[j];
33
34             if(sum > x)
35             {
36                 int length = j - i + 1;
37
38                 if(length < minLength)
39                 {
40                     minLength = length;
41                 }
42
43                 break;
44             }
45         }
46     }
47
48     if(minLength == n + 1)
49     {
50         minLength = 0;
51     }
52
53     printf("\nSmallest subarray length: %d\n", minLength);
54
55     return 0;
56 }
```

Output:

```
Roll Number : CH.EN.U4CCE24020
```

```
Enter the size of array: 5
```

```
Enter the array elements:
```

```
36
```

```
25
```

```
31
```

```
5
```

```
14
```

```
Enter the value of x: 2
```

```
Smallest subarray length: 1
```

```
-----
```

```
Process exited after 18.18 seconds with return value 0
```

```
Press any key to continue . . .
```

15. Given a sorted array `arr[]` with possibly some duplicates, find the first and last occurrences of an element `x` in the given array.

Note: If the number `x` is not found in the array then return both the indices as -1.

Examples:

Input: `arr[] = [1, 3, 5, 5, 5, 5, 67, 123, 125]`, `x = 5`

Output: `[2, 5]`

Explanation: First occurrence of 5 is at index 2 and last occurrence of 5 is at index 5

Input: `arr[] = [1, 3, 5, 5, 5, 5, 7, 123, 125]`, `x = 7`

Output: `[6, 6]`

Explanation: First and last occurrence of 7 is at index 6

Input: `arr[] = [1, 2, 3]`, `x = 4`

Output: `[-1, -1]`

Explanation: No occurrence of 4 in the array, so, output is `[-1, -1]`

Constraints:

$1 \leq \text{arr.size()} \leq 10^6$

$1 \leq \text{arr}[i], x \leq 10^9$

Code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int n, x;
6
7      printf("Roll Number : CH.EN.U4CCE24020\n\n");
8
9      printf("Enter the size of array: ");
10     scanf("%d", &n);
11
12     int arr[n];
13
14     printf("Enter the sorted array elements:\n");
15
16     for(int i = 0; i < n; i++)
17     {
18         scanf("%d", &arr[i]);
19     }
20
21     printf("Enter the element to search: ");
22     scanf("%d", &x);
23
24     int first = -1;
25     int last = -1;
26
27     for(int i = 0; i < n; i++)
28     {
29         if(arr[i] == x)
30         {
31             if(first == -1)
32             {
33                 first = i;
34             }
35
36             last = i;
37         }
38     }
39
40     printf("\nOutput: [%d, %d]\n", first, last);
41
42     return 0;
43 }
```

Output:

```
Roll Number : CH.EN.U4CCE24020

Enter the size of array: 3
Enter the sorted array elements:
52
64
84
Enter the element to search: 52

Output: [0, 0]

-----
Process exited after 12.5 seconds with return value 0
Press any key to continue . . . |
```

16. Given a Binary Search Tree (BST) and a range l-h (inclusive), your task is to return the number of nodes in the BST whose value lie in the given range.

Examples :

Input: root[] = [10, 5, 50, 1, N, 40, 100], l = 5, h = 45

Output: 3

Explanation: There are three nodes in range [5, 45] = 5, 10 and 40.

Input: root[] = [10, 5, 50, 1, N, 40, 100], l = 10, h = 100

Output: 4

Explanation: There are four nodes in range [10, 100] = 10, 40, 50 and 100.

Input: root[] = [1, 2, 3], l = 23, h = 95

Output: 0

Explanation: There are no nodes in range [23, 95].

Constraints:

$1 \leq \text{Number of nodes} \leq 10^5$

$1 \leq l \leq h \leq 10^5$

Code:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node
5  {
6      int data;
7      struct Node *left;
8      struct Node *right;
9  };
10
11 struct Node* createNode(int value)
12 {
13     struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
14
15     newNode->data = value;
16     newNode->left = NULL;
17     newNode->right = NULL;
18
19     return newNode;
20 }
21
22 struct Node* insert(struct Node *root, int value)
23 {
24     if(root == NULL)
25     {
26         return createNode(value);
27     }
28     if(value < root->data)
29     {
30         root->left = insert(root->left, value);
31     }
32     else
33     {
34         root->right = insert(root->right, value);
35     }
36
37     return root;
38 }
39
40 int countNodesInRange(struct Node *root, int l, int h)
41 {
42     if(root == NULL)
43     {
44         return 0;
45     }
46
47     int count = 0;
48
49     if(root->data >= l && root->data <= h)
50     {
51         count = 1;
52     }
53
54     return count +
55         countNodesInRange(root->left, l, h) +
56         countNodesInRange(root->right, l, h);
57 }
58
59 int main()
60 {
61     int n, l, h;
62
63     printf("Roll Number : CH.EN.U4CCE24020\n\n");
64
65     printf("Enter number of nodes: ");
66     scanf("%d", &n);
67
68     struct Node *root = NULL;
69
70     printf("Enter BST node values:\n");
71
72     for(int i = 0; i < n; i++)
73     {
74         int value;
75         scanf("%d", &value);
76
77         root = insert(root, value);
78     }
79
80     printf("Enter lower range value: ");
81     scanf("%d", &l);
82
83     printf("Enter higher range value: ");
84     scanf("%d", &h);
85
86     int result = countNodesInRange(root, l, h);
87
88     printf("\nNumber of nodes in range [%d, %d]: %d\n", l, h, result);
89
90     return 0;
91 }

```

Output:

```
Roll Number : CH.EN.U4CCE24020

Enter number of nodes: 3
Enter BST node values:
2
5
4
Enter lower range value: 1
Enter higher range value: 9

Number of nodes in range [1, 9]: 3

-----
Process exited after 13.67 seconds with return value 0
Press any key to continue . . . |
```


17. Given an array `arr[]` of integers, determine whether it contains a triplet whose sum equals zero. Return true if such a triplet exists, otherwise, return false.

Examples:

Input: `arr[] = [0, -1, 2, -3, 1]`

Output: true

Explanation: The triplet `[0, -1, 1]` has a sum equal to zero.

Input: `arr[] = [1, 2, 3]`

Output: false

Explanation: No triplet with a sum of zero exists.

Input: `arr[] = [-5, 3, 2, -1, 0, 1]`

Output: true

Explanation: The triplet `[-5, 3, 2]` has a sum equal to zero.

Constraints:

$1 \leq \text{arr.size()} \leq 10^3$

$-10^6 \leq \text{arr}[i] \leq 10^6$

Code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int n;
6
7      printf("Roll Number : CH.EN.U4CCE24020\n\n");
8
9      printf("Enter the size of array: ");
10     scanf("%d", &n);
11
12     int arr[n];
13
14     printf("Enter the array elements:\n");
15
16     for(int i = 0; i < n; i++)
17     {
18         scanf("%d", &arr[i]);
19     }
20
21     int found = 0;
22
23     // Check all possible triplets
24     for(int i = 0; i < n - 2; i++)
25     {
26         for(int j = i + 1; j < n - 1; j++)
27         {
28             for(int k = j + 1; k < n; k++)
29             {
30                 if(arr[i] + arr[j] + arr[k] == 0)
31                 {
32                     found = 1;
33                     break;
34                 }
35             }
36
37             if(found == 1)
38             {
39                 break;
40             }
41         }
42
43         if(found == 1)
44         {
45             break;
46         }
47     }
48
49     if(found == 1)
50     {
51         printf("\nOutput: true\n");
52     }
53     else
54     {
55         printf("\nOutput: false\n");
56     }
57
58     return 0;
59 }
```

Output:

```
Roll Number : CH.EN.U4CCE24020
```

```
Enter the size of array: 5
```

```
Enter the array elements:
```

```
2
```

```
1
```

```
6
```

```
5
```

```
4
```

```
Output: false
```

```
-----
```

```
Process exited after 13.31 seconds with return value 0
```

```
Press any key to continue . . .
```

18. Given a binary matrix `mat[][]` of dimensions $N \times M$ such that 1 denotes land and 0 denotes water. Find the number of closed islands in the given matrix.

An island is a 4-directional(up,right,down and left) connected part of 1's.

Note: A closed island is a group of 1s surrounded by only 0s on all the boundaries (except diagonals). In simple words, a closed island is an island whose none of the 1s lie on the edges of the matrix.

Example 1:

Input:

$N = 5, M = 8$

```
mat[][] = { {0, 0, 0, 0, 0, 0, 0, 1},
             {0, 1, 1, 1, 1, 0, 0, 1},
             {0, 1, 0, 1, 0, 0, 0, 1},
             {0, 1, 1, 1, 1, 0, 1, 0},
             {1, 0, 0, 0, 0, 1, 0, 1} }
```

Output:

2

Explanation:

```
mat[][] = { {0, 0, 0, 0, 0, 0, 0, 1},
             {0, 1, 1, 1, 1, 0, 0, 1},
             {0, 1, 0, 1, 0, 0, 0, 1},
             {0, 1, 1, 1, 1, 0, 1, 0},
             {1, 0, 0, 0, 0, 1, 0, 1} }
```

There are 2 closed islands. The islands in dark are closed because they are completely surrounded by 0s (water). There are two more islands in the last column of the matrix, but they are not completely surrounded by 0s. Hence they are not closed islands.

Example 2:

Input:

$N = 3, M = 3$

```
mat[][] = { {1, 0, 0},
             {0, 1, 0},
             {0, 0, 1} }
```

Output:

1

Explanation:

```
mat[][] = { {1, 0, 0},
             {0, 1, 0},
             {0, 0, 1} }
```

There is just a one closed island.

Your task:

You don't need to read input or print anything. Your task is to complete the function `closedIslands()` which takes two integers N and M , and a 2D binary matrix `mat` as input parameters and returns the number of closed islands.

Expected Time Complexity: $O(N*M)$ Expected Auxiliary Space: $O(N*M)$

Constraints:

 $1 \leq N, M \leq 500$ **Code:**

```

1  #include <iostream>
2
3  int N, P;
4
5  void dfs(int mat[500][500], int visited[500][500], int i, int j, int *isClosed)
6  {
7      if(i < 0 || j < 0 || i >= N || j >= P)
8      {
9          return;
10     }
11
12     if(mat[i][j] == 0 || visited[i][j] == 1)
13     {
14         return;
15     }
16
17     visited[i][j] = 1;
18
19     // If island touches boundary, it is not closed
20     if(i == 0 || j == 0 || i == N - 1 || j == P - 1)
21     {
22         *isClosed = 0;
23     }
24
25     dfs(mat, visited, i + 1, j, isClosed);
26     dfs(mat, visited, i - 1, j, isClosed);
27     dfs(mat, visited, i, j + 1, isClosed);
28     dfs(mat, visited, i, j - 1, isClosed);
29 }
30
31 int main()
32 {
33     printf("Roll Number : CN.EN.SACCE240204\n");
34     printf("Enter number of rows and columns: ");
35     scanf("%d %d", &N, &P);
36
37     int mat[500][500];
38     int visited[500][500] = {0};
39
40     printf("Enter matrix elements:\n");
41
42     for(int i = 0; i < N; i++)
43     {
44         for(int j = 0; j < P; j++)
45         {
46             scanf("%d", &mat[i][j]);
47         }
48     }
49
50     int closedIslands = 0;
51
52     for(int i = 0; i < N; i++)
53     {
54         for(int j = 0; j < P; j++)
55         {
56             if(mat[i][j] == 1 && visited[i][j] == 0)
57             {
58                 int isClosed = 1;
59
60                 dfs(mat, visited, i, j, &isClosed);
61
62                 if(isClosed == 1)
63                 {
64                     closedIslands++;
65                 }
66             }
67         }
68     }
69
70     printf("\nNumber of closed islands: %d\n", closedIslands);
71
72     return 0;
73 }
74

```

Output:

```
Roll Number : CH.EN.U4CCE24020

Enter number of rows and columns: 2
3
Enter matrix elements:
5
8
6
3
1
0

Number of closed islands: 0

-----
Process exited after 7.764 seconds with return value 0
Press any key to continue . . .
```

19. You are given a histogram represented by an array `arr[ ]`, where each element of the array denotes the height of the bars in the histogram. All bars have the same width of 1 unit.

Your task is to find the largest rectangular area possible in the given histogram, where the rectangle can be formed using a number of contiguous bars.

Examples:

Input: `arr[] = [60, 20, 50, 40, 10, 50, 60]`

Largest-Rectangular-Area-in-a-Histogram

Output: 100

Explanation: We get the maximum by picking bars highlighted above in green (50, and 60). The area is computed (smallest height) * (no. of the picked bars) = $50 * 2 = 100$.

Input: `arr[] = [3, 5, 1, 7, 5, 9]`

Output: 15

Explanation: We get the maximum by picking bars 7, 5 and 9. The area is computed (smallest height) * (no. of the picked bars) = $5 * 3 = 15$.

Input: `arr[] = [3]`

Output: 3

Explanation: In this example the largest area would be 3 of height 3 and width 1.

Constraints:

$1 \leq \text{arr.size()} \leq 10^5$

$0 \leq \text{arr}[i] \leq 10^4$

Code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int n;
6
7      printf("Roll Number : CH.EN.U4CCE24020\n\n");
8
9      printf("Enter the number of histogram bars: ");
10     scanf("%d", &n);
11
12     int arr[n];
13
14     printf("Enter the heights of bars:\n");
15
16     for(int i = 0; i < n; i++)
17     {
18         scanf("%d", &arr[i]);
19     }
20
21     int maxArea = 0;
22
23     // Check all possible rectangles
24     for(int i = 0; i < n; i++)
25     {
26         int minHeight = arr[i];
27
28         for(int j = i; j < n; j++)
29         {
30             if(arr[j] < minHeight)
31             {
32                 minHeight = arr[j];
33             }
34
35             int width = j - i + 1;
36             int area = minHeight * width;
37
38             if(area > maxArea)
39             {
40                 maxArea = area;
41             }
42         }
43     }
44
45     printf("\nLargest Rectangular Area: %d\n", maxArea);
46
47     return 0;
48 }
```


Output:

```
Roll Number : CH.EN.U4CCE24020  
  
Enter the number of histogram bars: 2  
Enter the heights of bars:  
3  
2  
  
Largest Rectangular Area: 4  
  
-----  
Process exited after 5.58 seconds with return value 0  
Press any key to continue . . . |
```

20. Given a data stream `arr[]` where integers are read sequentially, the task is to determine the median of the elements encountered so far after each new integer is read.

The median is defined as follows:

1. Odd number of elements: The median is the middle element when the current set of numbers is sorted.
2. Even number of elements: The median is the arithmetic mean (average) of the two middle elements when the current set of numbers is sorted.

Examples:

Input: `arr[] = [5, 15, 1, 3, 2, 8]`

Output: `[5.0, 10.0, 5.0, 4.0, 3.0, 4.0]`

Explanation:

After reading 1st element of stream – 5 -> median = 5.0

After reading 2nd element of stream – 5, 15 -> median = $(5+15)/2 = 10.0$

After reading 3rd element of stream – 5, 15, 1 -> median = 5.0

After reading 4th element of stream – 5, 15, 1, 3 -> median = $(3+5)/2 = 4.0$

After reading 5th element of stream – 5, 15, 1, 3, 2 -> median = 3.0

After reading 6th element of stream – 5, 15, 1, 3, 2, 8 -> median = $(3+5)/2 = 4.0$

Input: `arr[] = [2, 2, 2, 2]`

Output: `[2.0, 2.0, 2.0, 2.0]`

Explanation:

After reading 1st element of stream – 2 -> median = 2.0

After reading 2nd element of stream – 2, 2 -> median = $(2+2)/2 = 2.0$

After reading 3rd element of stream – 2, 2, 2 -> median = 2.0

After reading 4th element of stream – 2, 2, 2, 2 -> median = $(2+2)/2 = 2.0$

Constraints:

$1 \leq \text{arr.size()} \leq 10^5$

$1 \leq \text{arr}[i] \leq 10^6$

Code:

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int n;
6
7      printf("Roll Number : CH.EN.U4CCE24020\n\n");
8
9      printf("Enter the number of elements: ");
10     scanf("%d", &n);
11
12     int arr[n];
13     int temp[n];
14
15     printf("Enter the array elements:\n");
16
17     for(int i = 0; i < n; i++)
18     {
19         scanf("%d", &arr[i]);
20     }
21
22     printf("\nMedian after each insertion:\n");
23
24     for(int i = 0; i < n; i++)
25     {
26         // Copy elements till current index
27         for(int j = 0; j <= i; j++)
28         {
29             temp[j] = arr[j];
30         }
31
32         // Sort current elements
33         for(int j = 0; j <= i; j++)
34         {
35             for(int k = j + 1; k <= i; k++)
36             {
37                 if(temp[j] > temp[k])
38                 {
39                     int swap = temp[j];
40                     temp[j] = temp[k];
41                     temp[k] = swap;
42                 }
43             }
44         }
45
46         float median;
47
48         // Odd number of elements
49         if((i + 1) % 2 != 0)
50         {
51             median = temp[(i + 1) / 2];
52         }
53         // Even number of elements
54         else
55         {
56             // Odd number of elements
57             if((i + 1) % 2 != 0)
58             {
59                 median = temp[(i + 1) / 2];
60             }
61             // Even number of elements
62             else
63             {
64                 median = (temp[i / 2] + temp[(i / 2) + 1]) / 2.0;
65             }
66         }
67
68         printf("%.1f ", median);
69
70         printf("\n");
71
72         return 0;
73     }

```

Output:

```
Roll Number : CH.EN.U4CCE24020

Enter the number of elements: 6
Enter the array elements:
2
5
2
1
3
0

Median after each insertion:
2.0 3.5 2.0 2.0 2.0 2.0

-----
Process exited after 10.36 seconds with return value 0
Press any key to continue . . .
```